

Построение пайплайнов данных

Helm, CI/CD

Малахов Дмитрий Андреевич
Новосельцев Максим Сергеевич

23 января 2026 года

Helm-чарт

Helm — это не просто команда для установки приложений в Kubernetes. Это инструмент управления приложениями, где чарт является его фундаментом.

Чарт — это единица управления, которая инкапсулирует версию, конфигурацию и зависимости вашего приложения.

Структура

helm create

```
my-chart/  
├─ Chart.yaml  
├─ values.yaml  
├─ charts/  
├─ templates/  
│   ├─ deployment.yaml  
│   ├─ service.yaml  
│   ├─ ingress.yaml  
│   └─ NOTES.txt  
└─ tests/  
    └─ .helmignore
```

Структура

Chart.yaml

```
apiVersion: v2
name: my-nginx
version: surveyed
appVersion: "1.21.1"
description: A minimal Nginx deployment chart.
type: application
dependencies:
  - name: redis
    version: "~12.0.0"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
```

Структура

values.yaml

```
# Количество реплик пода
replicaCount: 1

# Настройки образа контейнера
image:
  repository: nginx
  tag: "1.21.1"
  pullPolicy: IfNotPresent

# Настройки сервиса
service:
  type: ClusterIP
  port: 80
```

Директория templates/

Типовой набор включает:

- deployment.yaml: Шаблон для развертывания (Deployment).
- service.yaml: Шаблон для сервиса (Service).
- configmap.yaml, secret.yaml: Для конфигураций и секретов.
- NOTES.txt: Шаблон для вывода кастомного сообщения после установки.

Доступ к values

```
# Доступ к значениям
{{ .Values.key }}
{{ .Values.nested.key }}

# Функции
{{ .Values.image | default "nginx:latest" }}
{{ .Values.name | upper }}
{{ .Values.replicas | toJson }}

# Логика
{{- if .Values.ingress.enabled }}
...
{{- end }}

{{- if eq .Values.environment "production" }}
...
{{- end }}

# Циклы
{{- range .Values.ingress.hosts }}
- host: {{ .host }}
{{- end }}
```

upgrade

Для обновления существующего релиза используйте `helm upgrade`

```
# Обновление релиза с новыми параметрами
helm upgrade myapp ./myapp \
  --namespace myapp-ns \
  --values values-staging.yaml \
  --set image.tag="1.1.0"
```


Откатить неудачный релиз

```
# Просмотр истории релизов  
helm history myapp --namespace myapp-ns  
  
# Откат к предыдущей версии  
helm rollback myapp 1 --namespace myapp-ns
```

Kustomize

Kustomize — инструмент для кастомизации Kubernetes-манифестов без шаблонизации. Вместо Go-шаблонов с подстановками он работает с обычными YAML-файлами и накладывает на них патчи.

Начиная с kubectl 1.14 Kustomize встроен в kubectl: команда `kubectl apply -k` применяет манифесты с кастомизацией без установки дополнительного ПО.

base/overlays

app/

base/

deployment.yaml # базовый манифест

service.yaml

kustomization.yaml # список ресурсов

overlays/

dev/

kustomization.yaml # патч: replicas: 1

staging/

kustomization.yaml # патч: replicas: 2

prod/

kustomization.yaml # патч: replicas: 3

Выбирайте Helm, когда:

- Устанавливаете стороннее ПО
- Распространяете приложение
- Нужен rollback
- Требуется сложная параметризация

Выбирайте Kustomize, когда:

- Используете внутренние микросервисы
- GitOps как основа
- Нужно модифицировать чужой манифест

Helm + Kustomize вместе

- Post-rendering: Helm рендерит, Kustomize патчит

```
helm install my-release ./my-chart --post-renderer ./kustomize-wrapper.sh
```

```
#!/bin/bash
cat <&0 > all.yaml
kustomize build .
rm all.yaml
```

Helm + Kustomize вместе

- Kustomize как драйвер через helmCharts
В kustomization.yaml можно указать Helm-чарт как источник

```
helmCharts:
```

```
  - name: cert-manager  
    repo: https://charts.jetstack.io  
    version: v1.14.0  
    valuesFile: values.yaml
```

Helm + Kustomize ВМесте

- helm template + kustomize build

```
helm template my-release ./chart > base/rendered.yaml  
kubectl apply -k overlays/prod/
```


GitOps

GitOps — это способ управлять конфигурацией и деплоем через Git, а не через ручные действия в кластере.

- Он не заменяет CI/CD: CI собирает артефакт, а GitOps следит, чтобы среда реально пришла к состоянию из репозитория.
- На практике GitOps чаще всего встречается в Kubernetes-связке с Argo CD или Flux CD, Helm и Kustomize.
- Главная ценность GitOps — прозрачная история изменений, воспроизводимость и меньше ручных правок в production.
- Если у команды ещё нет Docker, CI/CD, review-процесса и понятной структуры конфигурации, GitOps внедрять рано.

Как выглядит GitOps-цикл

1. Разработчик меняет конфигурацию в Git
2. Изменение проходит review и merge
3. Контроллер читает репозиторий
4. Система сравнивает Git и кластер
5. Среда приводится к нужной версии

ArgoCD

 **argo**
v3.2.5+56f440

 Applications

 Settings

 User Info

 Documentation

Application filters

☐ ★ Favorites Only

SYNC STATUS

☐ ⚪ Unknown 0

☒ 🟢 Synced 49

☐ 🟡 OutOfSync 9

HEALTH STATUS

☐ 🔵 Progressing 2

☐ ⏸ Suspended 0

☒ 🟢 Healthy 54

☐ 🟠 Degraded 0

☐ 🟡 Missing 2

☐ 🟡 Unknown 0

LABELS

LABELS

PROJECTS

PROJECTS

CLUSTERS

CLUSTERS

NAMESPACES

Applications

+ NEW APP

↻ SYNC APPS

🔄 REFRESH APPS

Q Search applications...

   Log out

Destinat... in-cluster
Namesp... argowf
Created ... 04/23/2026 13:37:42 (4 hours ago)
Last Syn... 04/23/2026 13:50:39 (4 hours ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

Destinat... in-cluster
Namesp... cnpg-system
Created ... 04/02/2026 15:02:12 (21 days ago)
Last Syn... 04/02/2026 20:58:43 (21 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

Destinat... in-cluster
Namesp... infiscal
Created ... 04/02/2026 14:43:22 (21 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

Destinat... in-cluster
Namesp... keel
Created ... 04/02/2026 15:10:41 (21 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

🔍 monitoring ☆

Project: default
Labels:
Status: 🟢 Healthy 🟢 Synced
Reposit... https://gitlab.consultant.ru/dad/infrastr...
Target R... HEAD
Path: applications/monitoring
Destinat... in-cluster
Namesp... monitoring
Created ... 04/02/2026 19:48:31 (21 days ago)
Last Syn... 04/06/2026 12:04:02 (17 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

🔍 nessie ☆

Project: default
Labels:
Status: 🟢 Healthy 🟢 Synced
Reposit... https://gitlab.consultant.ru/dad/infrastr...
Target R... HEAD
Path: applications/nessie
Destinat... in-cluster
Namesp... nessie-ns
Created ... 04/01/2026 21:03:57 (22 days ago)
Last Syn... 04/01/2026 21:04:19 (22 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

🔍 ozone ☆

Project: default
Labels:
Status: 🟢 Healthy 🟡 OutOfSync
Reposit... https://gitlab.consultant.ru/dad/infrastr...
Target R... ozone
Path: applications/ozone
Destinat... in-cluster
Namesp... ozone
Created ... 04/06/2026 15:06:23 (17 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

🔍 pgadmin ☆

Project: default
Labels:
Status: 🟢 Healthy 🟢 Synced
Reposit... https://gitlab.consultant.ru/dad/infrastr...
Target R... HEAD
Path: applications/pgadmin
Destinat... in-cluster
Namesp... pgadmin
Created ... 04/02/2026 14:54:50 (21 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

🔍 rbac-wizard ☆

Project: default
Labels:
Status: 🟢 Healthy 🟢 Synced
Reposit... https://gitlab.consultant.ru/dad/infrastr...
Target R... HEAD
Path: applications/rbac-wizard
Destinat... in-cluster
Namesp... rbac-wizard
Created ... 04/02/2026 15:04:19 (21 days ago)
Last Syn... 04/02/2026 17:18:22 (21 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

🔍 redis ☆

Project: default
Labels:
Status: 🟢 Healthy 🟢 Synced
Reposit... https://gitlab.consultant.ru/dad/infrastr...
Target R... HEAD
Path: applications/redis
Destinat... in-cluster
Namesp... redis
Created ... 04/02/2026 15:06:36 (21 days ago)
Last Syn... 04/02/2026 17:20:42 (21 days ago)

↻ SYNC

🔄 REFRESH

🗑 DELETE

Previous 1 2 3 4 5 6 Next

Sort: name ▾ Items per page: 10 ▾

Домашнее задание

Вам необходимо спроектировать структуру репозитория (или нескольких репозиториев) и описать полный CI/CD пайплайн для развертывания разнородных приложений в Kubernetes: Spark-приложения (обработка данных), сторонних продуктов из Helm (например, установка самого Argo Workflows, Spark Operator, MinIO, PostgreSQL) и самописных витрин для отображения данных (Node.js/React).

1. Выберите технологический стек для каждого типа приложений:

1. Определитесь где будет собираться Docker-образ всех приложений (ArgoWF, GitLab CI, Jenkins и т.д.)
2. Где будут храниться артефакты (Nexus, artifact keeper, jfrog Artifactory и т.д.).

2. Спроектируйте структуру репозитория(ев): покажите, где лежат:

1. Helm-чарты сторонних продуктов (Argo Workflows, MinIO и т.д.);
2. YAML-файлы/Helm-чарты внутренних витрин для отображения данных
3. Kustomize-оверлеи для разных сред (или без них);
4. Универсальные Argo Workflow-темплейты (из прошлого ДЗ);
5. Исходники для Spark-приложений и React-витрин;
6. Argo Workflow-файлы для запуска джоб

3. Опишите пайплайн от `git push` до запуска в кластере:

1. Сборка образа Spark-приложения и публикация в registry.
2. Создание/генерация Argo Workflow-файла (YAML), который запускает Spark-джобу с использованием ваших универсальных темплейтов.
3. Где и как хранится этот workflow-файл (в том же репозитории или в отдельном).
4. Как и откуда Argo Workflows его подхватывает и запускает.
5. Как применяются Helm/Kustomize для деплоя вспомогательных сервисов.

4. Обоснуйте свой выбор:

Какие преимущества вашей схемы с точки зрения переиспользования workflow-темплейтов, версионирования артефактов, изоляции конфигураций сред и упрощения отката.